# CS 623 Assignment 2

Roll No. 230217

## Setup

All numbers in this report were measured on the GPU3 department server: NVIDIA A40 (compute capability 8.6, 46 GB), CUDA 12.6, built with `nvcc -O2 -std=c++17 -arch=sm_86` (Problem 2 adds `-fmad=false` so that the GPU sums round exactly like the CPU code; Problem 3 adds `-lineinfo` for the profiler). The GPU was shared: another user's job kept it busy (about 90% utilisation in `nvidia-smi`) during the runs, so timings depend on what else is running. I therefore repeated each measurement several times at different times of day and report the *best* time (contention can only add time) together with the worst time seen. For the interleaved Problem 2 runs I logged `nvidia-smi` before each run, and the best times were reproduced in runs that started with no other process on the GPU. All CUDA allocations, copies, memsets, kernel launches and synchronisations in Problems 2 and 3 are checked with the `cudaCheckError` macro of the Problem 1 template (launches through `cudaGetLastError` and the following synchronising call).

**Build and run.** From the `230217-assign2` directory:

```
make                        # problem1, problem2 (v1,v2,v3), problem3 (2 variants)
./problem1/problem1.out
cd problem2 && ./problem2-v1.out && ./problem2-v2.out && ./problem2-v3.out
cd ../problem3 && ./problem3-vanilla.out N $PWD/insert_100000.bin \
                  $PWD/search_100000.bin $PWD/delete_100000.bin
```

Problem 1 takes its size from the constant `Input_Width` in `230217-prob1.cu` (`1 << 6`, `1 << 7` or `1 << 8`); change it and rebuild with `make problem1`. Problem 2 must be run from inside `problem2/`, because it opens `disp.txt` and `grid.txt` from the current directory. The three versions of Problem 2 are one source file, `230217-prob2.cu`, built with `-DVERSION=1`, `2` and `3`; the two variants of Problem 3 are one source file built with and without `-DBLOCKED`.

## Problem 1: 3D convolution

### Implementation

The input is an $N \times N \times N$ array of `float`, the filter is a $5 \times 5 \times 5$ cube, and cells outside the array are treated as zero. Every version computes

$$\text{out}[z][y][x] = \sum_{f_z, f_y, f_x = 0}^{4} \text{filter}[f_z][f_y][f_x] \cdot \text{in}[z + f_z - 2][y + f_y - 2][x + f_x - 2].$$

- **CPU:** single-threaded reference with six nested loops.

- **Basic kernel:** one thread per output cell, $8 \times 8 \times 8$ blocks and an $(N/8)^3$ grid. The input and the filter are read from global memory, up to 125 reads of each per thread.
- **Optimised kernel:** the same mapping with (a) a $12 \times 12 \times 12$ shared-memory tile per block ($8 + 2 \cdot 2$ in each dimension, ghost cells set to zero), loaded cooperatively by the 512 threads followed by `__syncthreads()`; (b) the filter in `__constant__` memory; (c) the innermost filter loop unrolled by hand.

Each kernel is run once as a warm-up and then 10 times; the program reports the average. *Kernel time* uses CUDA events around the launch; *end-to-end time* uses `chrono` around the host-to-device copy, the kernel and the device-to-host copy. `check_result` compares every GPU output with the CPU output: it reported "No differences found" for both kernels at all three sizes, both with the template's all-ones input and with random integer input (so that indexing errors cannot hide behind a constant input).

## Results

Times in ms. Each entry is the best of 6 program runs (made in two sessions); every program run already averages 10 kernel launches. The ranges over all runs are given in the second table.

| $N$ | CPU | Basic kernel | Basic end-to-end | Opt kernel | Opt end-to-end |
|---|---|---|---|---|---|
| 64 | 38 | 0.047 | 0.80 | 0.026 | 0.76 |
| 128 | 337 | 0.293 | 4.19 | 0.143 | 4.00 |
| 256 | 2754 | 2.321 | 26.94 | 1.106 | 24.48 |

Range over all runs (ms)

| $N$ | CPU | Basic kernel | Basic end-to-end | Opt kernel | Opt end-to-end |
|---|---|---|---|---|---|
| 64 | 38–60 | 0.047–0.049 | 0.80–1.40 | 0.026–0.032 | 0.76–1.41 |
| 128 | 337–364 | 0.293–0.302 | 4.19–7.10 | 0.143–0.156 | 4.00–4.66 |
| 256 | 2754–2933 | 2.321–3.305 | 26.94–42.10 | 1.106–1.120 | 24.48–37.32 |

| $N$ | Basic kernel vs CPU | Opt kernel vs CPU | Opt kernel vs basic kernel |
|---|---|---|---|
| 64 | 817× | 1473× | 1.80× |
| 128 | 1152× | 2351× | 2.04× |
| 256 | 1186× | 2490× | 2.10× |

## Discussion

In the basic kernel every input value is fetched from global memory by up to 125 different threads, and every thread also reads the same 125 filter weights from global memory. The optimised kernel removes this redundancy: a block computes an $8^3$ output chunk from a $12^3$ input region that is loaded from global memory once (1728 reads instead of about $512 \times 125$) and then reused from shared memory, the filter reads become constant-cache broadcasts (all threads of a warp read the same weight at the same time), and the unrolled inner loop gives the compiler more independent loads and multiplies to schedule. The measured gain of the optimised kernel over the basic one grows with the problem size, from 1.80× at $N = 64$ to 2.10× at $N = 256$: at small sizes the kernel is so short (tens of microseconds) that launch and scheduling overheads probably limit both versions (I did not profile this), while at $N = 256$ the memory traffic saved by the tile matters.

The end-to-end numbers tell a different story. For $N = 256$ the end-to-end time of the basic kernel is about 12 times its kernel time, and the optimised kernel's end-to-end time is only slightly lower than the basic one's (the end-to-end times also vary the most between runs, see the table of ranges): copying the $256^3$-float input and output over PCIe takes far longer than either kernel, so a faster kernel barely changes the total unless the data stays on the device across many launches.

# Problem 2: grid search

## Mapping the iteration space

The grid has 10 variables with 13 values each ($0.5, \ldots, 6.5$ from `grid.txt`; the count is computed as $\lfloor(\text{end} - \text{start})/\text{step}\rfloor$ from the file, never hard-coded), so $13^{10} \approx 1.4 \times 10^{11}$ points. A point is a result if $|\sum_j c_{ij}x_j - d_i| \le e_i$ for all 10 rows ($e_i = 0.3 \cdot ey_i$). A point is identified by one number in mixed radix, `point_id` $= ((i_1 n_2 + i_2)n_3 + \ldots)n_{10} + i_{10}$, stored as a 64-bit integer because $13^{10} > 2^{32}$. Ascending `point_id` is exactly the loop order of `problem2-v0.c`, so the matches are sorted on the host and written in the same order and format as the reference. The coefficients, bounds, steps and counts are copied to `__constant__` memory.

## The three versions

**(i) Simple port.** The host loops over variables 1 and 2 (169 kernel launches); each launch has one thread per combination of variables 3–6 (28 561 threads), and each thread loops over variables 7–10. All ten row sums are computed from scratch for every point. A thread that finds a match takes a slot with `atomicAdd` on a counter and stores the `point_id`; the host reads the counter after each launch, copies the matches back and finally sorts them.

**(ii) Optimised.** The same exhaustive search with the basic optimisations of the cascaded partial-sum structure of my own earlier CPU version of this search: (a) *reuse of partial sums* — a table of running sums per row is kept, so changing the innermost variable costs one multiply-add per row instead of ten, and $d_i$ is folded into the first level; (b) *early exit on the final check only* — the last row test stops at the first row that fails; (c) *fewer, larger kernel launches* — the host loops only over variable 1 (13 launches) and each launch has $13^5 = 371\,293$ threads, which fills the GPU much better than the 28 561 threads of (i). No point is skipped: every one of the $1.4 \times 10^{11}$ points is still checked.

**(iii) Thrust.** The point numbers are processed in slices of $2^{28}$. For each slice a `thrust::counting_iterator` feeds `thrust::transform`, whose functor decodes the point number, evaluates the ten rows (returning 0 at the first failing row) and writes a flag; `thrust::count_if` sizes the output and `thrust::copy_if` with the flags as stencil collects the matching point numbers. The slices go in increasing order, so the output is already sorted.

Shared-memory tiling and prefetching, which the assignment lists as examples, are not used: the only data are the 120+30 numbers of the two input files (about 1.2 KB), which already live in constant memory and are read with the same address by all threads of a warp (a broadcast), so there is nothing to tile; I did not measure them and make no claim about them.

## Correctness

The CPU reference `problem2-v0.c` was also run on the full grid: it prints `result pnts: 11608` and its `results-v0.txt` is byte-identical to the result files of v1, v2 and v3. All results use the

same output format as `problem2-v0.c`. The files were compared with `diff`:

| Grid | Result points | Files compared |
|---|---|---|
| 6 values per variable ($6^{10}$ points) | 2071 | v0 = v1 = v2 = v3 (byte-identical) |
| 8 values per variable ($8^{10}$ points) | 9316 | v0 = v1 = v2 = v3 (byte-identical) |
| 13 values per variable (full grid) | 11608 | v0 = v1 = v2 = v3 (byte-identical) |

### Performance

Full grid, total time as printed by each program (host-side wall time of the whole search: kernel launches or Thrust calls plus copying the matches back; the final sort of the matches and the file output come afterwards and are not included). Each version was run 10 times, interleaved with the other versions, at different times while other users were also using the GPU. *Best* is the fastest run; it was reproduced in runs that started with `nvidia-smi` showing no other process on the GPU (utilisation 0–14%), to within 2% (e.g. version (ii) took 6.28 s in four separate runs). *Worst* is the slowest run, when another job was using the GPU at the same time. The CPU time is a single run of `problem2-v0.c` on the server.

| Version | Best (s) | Worst (s) | Speed-up vs (i) | Speed-up vs CPU |
|---|---|---|---|---|
| CPU `problem2-v0.c` | 676.2 | — | — | 1 |
| (i) simple port | 36.13 | 80.9 | 1.00 | 18.7 |
| (ii) optimised | 6.28 | 14.7 | 5.76 | 107.7 |
| (iii) Thrust | 34.89 | 91.2 | 1.04 | 19.4 |

### Discussion

Version (ii) is clearly the fastest, 5.76 times faster than (i): reusing the partial sums cuts the work per point from about 100 multiply-adds to a handful, the early exit avoids evaluating the remaining rows of a failing point (results are rare: 11608 of $1.4 \times 10^{11}$ points), and the larger launches keep the GPU better occupied. Versions (i) and (iii) are about equal on an idle GPU (36.13 s and 34.89 s). In the Thrust version every point is an independent element, so each element decodes its own 64-bit point number (ten 64-bit divisions and remainders, which are expensive on a GPU) and no work can be shared between neighbouring points, whereas (i) decodes a prefix once per thread and then runs through $13^4$ points; on the other hand (iii) stops at the first failing row and (i) does not. I did not profile either version, so this is my reading of the code, not a measurement of why they come out equal. Thrust gives short code, and `transform` plus a predicate is all the problem needs, but its element-wise model does not expose the loop structure that (ii) exploits. The effect of sharing the GPU is large: under load the times rose to 80.9 s, 14.7 s and 91.2 s, and (i) and (iii) changed places between runs, which is why only the idle-GPU best times are compared.

## Problem 3: counting Bloom filter

### Design

- **Structure.** The filter is a raw array of $N$ 8-bit counters on the GPU, stored as $\lceil N/4 \rceil$ 32-bit words. Counter `pos` is byte `pos % 4` of word `pos / 4`; `insert`, `delete` and `search` are three separate kernels with one thread per key, run one after the other from the host, so they never overlap. $k = 3$.

- **Atomics on bytes.** CUDA has no 8-bit atomics, so an increment is `atomicAdd(word, 1 << (8*(pos%4)))` (counter overflow is ignored, as allowed). A decrement uses an `atomicCAS` loop on the containing word that refuses to go below zero; a plain subtraction would borrow from the neighbouring counter when the byte is 0.
- **Delete.** Delete runs in two phases within one batch. A search kernel first records, for every key of the batch, whether all three of its counters are above 0, using the counters as they are before any decrement; then a delete kernel decrements the three counters (with the `atomicCAS` loop, never below 0) of the keys that were flagged. A key whose counters show it is absent therefore changes nothing. Taking the decision for all keys before any decrement makes the result independent of the order in which threads run. A key that was never inserted but whose three counters are all set by other keys (a false positive) cannot be told apart from a member by the filter and is decremented like one; this is an inherent limit of counting Bloom filters (see the experiment below), and the main traces delete only keys that were inserted.
- **Duplicates.** Each trace is sorted and de-duplicated on the host before it is copied to the GPU, so a key repeated within a trace increments its counters once. If a key were inserted in two different insert traces its counters would count both inserts; the test traces use one insert trace.
- **Hash functions.** `hash1`…`hash4` are separate `__host__ __device__` functions so that the host test computes exactly the same positions as the kernels. Each is scramble($key \cdot m + s$) with its own multiplier $m$ and seed $s$, where `scramble` xors the value with two rotations of itself (different rotation amounts per hash), multiplies by a second odd constant and xors with a right shift by 15. I checked them statistically (uniform over 128 and 1024 buckets, about 16 of 32 output bits flip per input bit flip, near-independent pairs).
- **Vanilla variant.** The three counters are `hash1,2,3(key) % N`.
- **Blocked variant.** The array is cut into 128-byte blocks (one L1 cache line); `hash4(key)` selects the block and `hash1,2,3(key) % 128` select the three counters inside it, so the $k+1 = 4$ hash functions of the assignment are all used and all counters of a key lie in one cache line. $N$ is rounded up to a multiple of 128. (The assignment names the first hash as the block selector; I use `hash4`, which gives the same structure.) If two of a key's counters coincide, that counter is incremented (and later decremented) once per hash, so insert and delete stay consistent.

### Traces and test routine

`trace_generator.cpp` writes three binary files of `uint32` keys for a given $n$ (no random-number generator: key $i$ is a fixed invertible scrambling of the number $i$, so keys for different numbers never coincide). The insert trace has keys $0..n-1$ with every tenth entry a repeat; the delete trace has every third inserted key; the search trace contains, in a shuffled order, a third keys that stay in the filter, a third keys that are deleted, and a third keys that were never inserted. The submission contains the traces for $n = 100\,000$ (110 000 / 100 000 / 33 334 keys, under 0.5 MB each).

`check_result()` keeps a host model: a plain byte array updated with the same rules (and the same `positions()` function) plus a `std::set` of the keys that should be present. After the insert and after the delete it copies the GPU counters back and counts how many differ from the model; after each search batch it compares every answer with the set and counts false negatives and false positives. The false-positive rate is the number of false positives divided by the number of all searches, as defined in the assignment. The set is used only in this test, never by the kernels.

### Results

On these traces, every run: 0 counters differ from the host model after insert and after delete, and 0 false negatives. Search trace of $n$ keys, a third of which were never inserted (so the rate among the keys that really are absent is three times the figure shown after the insert, and one and a half times after the delete, when two thirds of the keys are absent).

| Trace | $N$ | Variant | FP after insert | FP after delete |
|---|---|---|---|---|
| $n = 10^5$ (100k inserted) | $10^6$ | vanilla | 0.547% | 0.389% |
| | | blocked | 0.662% | 0.554% |
| $n = 2 \cdot 10^6$ | $2 \cdot 10^7$ | vanilla | 0.583% | 0.390% |
| | | blocked | 0.665% | 0.500% |
| $n = 10^5$ | $3 \cdot 10^5$ | vanilla | 8.42% | 7.73% |
| | | blocked | 8.52% | 7.91% |

The blocked variant has the higher false-positive rate in every row, as the assignment predicts: keys that fall into the same 128-counter block compete for those counters, and unevenly loaded blocks cost more false positives than lightly loaded blocks save. The gap is largest when the filter is lightly filled and shrinks as it fills up.

### Deleting keys that were never inserted

To test the limit described under *Delete*, I generated a delete trace in which 10 020 of its 43 354 keys were never inserted (generator option `ghost_pct` = 30) and ran it with $N = 3 \cdot 10^5$, so that many such keys look present. The insert and search traces are the $n = 10^5$ ones. Each program was run three times; the three runs gave identical output.

| Variant | Never-inserted keys that looked present | Counters differing from host model | False negatives after delete | FP rate after delete |
|---|---|---|---|---|
| vanilla | 2469 | 0 (all 3 runs) | 2025 | 6.77% |
| blocked | 2549 | 0 (all 3 runs) | 1989 | 6.90% |

The GPU counters match the sequential host model exactly. The false negatives are real members whose counters were decremented by never-inserted keys that looked present; no counting Bloom filter can avoid this, because the filter cannot tell such a key from a member. In an earlier version of the delete kernel, where each thread checked its counters and decremented them without waiting for the other threads, the same trace gave about 2 650 counters that differed from the host model, and a different number in every run (2657, 2659 and 2655); this is why delete now takes its decision for all keys of a batch before any decrement.

### Profiling with Nsight Compute

Profiled with Nsight Compute 2026.1 on the A40, for the first launch of each kernel on the $n = 2 \cdot 10^6$ traces with $N = 2 \cdot 10^7$ counters (2.2 million keys inserted, 2 million searched). Both variants were profiled.

| Metric | Vanilla insert | Vanilla search | Blocked insert | Blocked search |
|---|---|---|---|---|
| Kernel duration | 1.46 ms | 625 $\mu$s | 777 $\mu$s | 371 $\mu$s |
| DRAM traffic | 689 MB | 347 MB | 431 MB | 220 MB |
| Memory bandwidth utilisation (DRAM, % of peak) | 67.9 | 79.8 | 79.8 | 85.5 |
| Arithmetic intensity (FLOP/byte) | 0 | 0 | 0 | 0 |
| integer instructions per DRAM byte | 0.21 | 0.44 | 0.34 | 0.71 |
| Achieved occupancy (%) | 69.8 | 81.2 | 72.2 | 81.8 |
| Warp efficiency (active threads / 32) | 32/32 | 32/32 | 32/32 | 32/32 |
| L2 cache hit rate (%) | 9.7 | 10.4 | 36.5 | 19.9 |
| Long-scoreboard stalls (% of warp-active time) | 22.6 | 50.9 | 22.2 | 57.5 |

The kernels execute no floating-point instructions, so the arithmetic intensity in FLOP/byte is exactly zero; the integer-instruction rate per byte is given as the nearest meaningful figure. Warp efficiency is the `smsp__thread_inst_executed_per_inst_executed` ratio (32 means every lane is active), which is expected because the kernels have no data-dependent branches inside the warp except the final bounds check.

**Top three bottlenecks reported by the profiler** (in order of the profiler's estimated speed-up; it names the same causes for both variants).

1. **Uncoalesced global memory accesses.** Insert: 84% of the sectors are excess (estimated speed-up 83%, vanilla and blocked alike). Search: 83% of the sectors are excess (estimated speed-up 82% vanilla, 81% blocked), and the DRAM access-pattern check adds that only 5.1 of the 32 bytes of each fetched sector are used by a thread, with 90.5% (vanilla) / 81.2% (blocked) of those sectors missing in L2 (estimated speed-up 61% / 45%). Every key hashes to unrelated places, so neighbouring threads touch different cache lines.
2. **Memory-bound execution with long-scoreboard stalls.** Memory is busier than compute: DRAM throughput is 68–85% of peak, and a warp spends 288 (vanilla) / 202 (blocked) cycles per search waiting on an L1/global load (estimated speed-up 20% / 15%). The insert kernel moves about twice the DRAM traffic of the search kernel, presumably because each atomic is a read-modify-write.
3. **Low issue rate / occupancy.** Each scheduler issues an instruction only every 35–146 cycles with 0.01–0.04 eligible warps per cycle, because warps wait on memory, and achieved occupancy (70–84%) is below the theoretical 100% (estimated speed-up 15–32%).

Atomics are not named as a bottleneck by the profiler for these runs; their cost shows up in the insert kernel's higher DRAM traffic and longer duration.

**Effect of blocking.** The blocked variant is about 1.9× faster for insert and 1.7× for search, moves 36–37% less DRAM data, and has a higher L2 hit rate (36% / 20% against about 10%): the three counters of one key share a cache line, so one 128-byte line serves all three accesses. The price is the higher false-positive rate in the table above. Both kernels remain limited by uncoalesced access, since different keys still go to unrelated blocks.